

**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

CE/CZ4123/SC4023 BIG DATA MANAGEMENT

SEMESTER GROUP PROJECT

Group 38

Name	Matriculation Number
Tan Wei Yuan	U2122007H
Teo Jia Yun	U2120392J

Table Of Contents

Table Of Contents	2
1 Data Storage	3
1.1 Column Store Implementation	3
1.2 Data Columns Design for Efficient Processing	3
1.3 Read and Writing Input/Output Files	3
1.4 Exception Handling	3
2 Data Processing	4
3 Optimization	4
3.1 Query Filter Sequence	4
3.2 Shared Scan	4
3.3 Zone Maps	5
3.4 Bitmaps	5
4 Experiment Result	6
4.1 Query Execution Result	6
4.2 Optimization Result	7
5 Discussion & Conclusion	7
Appendix A: Data analysis for Query Sequencing	8
Appendix B: Experimentation Logs	9

1 Data Storage

1.1 Column Store Implementation

In-Memory vs Disk Storage

	In-Memory Storage	Disk-Based Storage
Implementation	Dictionary Keys → Column Name Value → Array of data	Binary files for each column
Persistence	Recreated every run	Created Once
Speed	Fast, no disk reads	Slower, reads from file
Scalability	Not suitable for large datasets	Scalable to large datasets

Based on the trade-offs from the table above, we have decided to use a Disk-Based storage approach.

To initialise the Column Store, the source data file called **ResalePricesSingapore.csv** is parsed to the class and the function `extract_and_store()` is called. The function will then proceed to read the data file using the pandas library where each column is iterated and stored as a separate binary file on disk. During the initialization, Zone maps and Bitmaps are also created to index different columns (refer to Section 3: Optimization)

1.2 Data Columns Design for Efficient Processing

Each columnar attribute in the source file is stored as a separate file on disk for efficient storage and retrieval. By storing each column separately, the column store is able to store each value in a column using suitable byte sizes for binary files. Specifically, columns with integer values are stored as 4-byte size while double values (decimal values) are stored as 8-byte sizes. This allows better storage efficiency rather than storing all values as strings with 50-byte sizes.

1.3 Read and Writing Input/Output Files

When reading and writing files, we used a generic `FileReader` to Open, Read and Write to our binary files. Based on the byte size of each column, we can correctly determine how to move the Reader to the next data item.

1.4 Exception Handling

Our application does simple exception handling, for the important rows, if there are null or incorrectly formatted values, the entire row is dropped. For other rows, a placeholder NA value is added. This minimizes the number of data items dropped.

2 Data Processing

We start by instantiating a **Query** class which takes in a matric number and breaks it down into **START_MONTH**, **END_MONTH** and **TOWN** based on the rules defined.

With the **Query** and **Column** Store ready, the **QueryProcessing** class takes in both classes and performs filtering by loading relevant column store files and applying the query filter. For example, to filter by time period, the class function will load the column store file called month.store and iterate through the values in the column store. Each value is checked for match with either **START_MONTH** or **END_MONTH**, and the positions of matching entries are recorded. With the filtered positions list, following queries will only load a subset of columnar data, requiring smaller I/O load and iteration count. When all filters are applied (**MONTH**, **TOWN**, **AREA**), the remaining positions are the entries that match the query parameters.

Once the final positions are obtained, a tuple reconstruction step is performed to get the resale prices and floor areas of the filtered entries. The data is then used to calculate the statistics using the Python Math library. All results are rounded to 2 decimal points.

3 Optimization

As the source data provided has already been sorted based on [month, town, resale_price] and indexed accordingly, the optimization to improve the Column Store query performance will be focused on the **Query Filter Sequence**, **Shared Scan**, **Zone Maps** and **Bit Maps**.

3.1 Query Filter Sequence

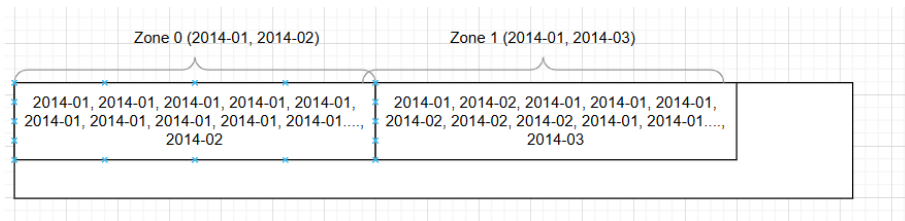
Column Store query leverages on the ability to load and filter by column, as well as selective loading by filtered positions. Hence, if the first filter is able to eliminate a large number of positions, the following filters would only need to load and iterate through a small subset of positions. This shows that the filter sequence is relevant to the performance of the query in the Column Store. To select the best filter sequence, we analyzed the distributions of each column shown in Appendix A. We observed that the column **month** has the lowest average number of transactions in each category, followed by **town** and lastly **floor_area**. This suggests that by applying the filter on month first, we will be able to filter out the largest number of possible positions on average. Hence, based on the logic, the best sequence of filters would be month, followed by town and lastly floor_area.

3.2 Shared Scan

The idea of shared scan is to reduce the number of repetitive iterations and instead run the filters over a single scan over the column. This can be achieved by loading all the necessary columns and applying the filters on respective columnar data within one pass. Hence, shared scan design aims to improve the query execution efficiency by needing to iterate through the column once instead of multiple times.

3.3 Zone Maps

Zone mapping is an efficient indexing technique used in columnar storage systems to reduce unnecessary data scans during query processing. The columns are divided into fixed-size virtual blocks, and for every block, the minimum and maximum values are recorded as metadata. Instead of scanning the entire column, only the relevant zones are loaded, significantly reducing the number of positions that need to be iterated over.



Implementation

Although the optimal zone size is typically tied to the CPU's block size for cache efficiency, this project uses a fixed logical block size of 1000 values per zone for simplicity. While zonemaps can be created for most columns containing sequential data, for the project we would only apply on the first filter, which is the **Month** column.

Our processor first scans the zone map, to identify the zones where the target month lies within the min and max value. Given the zone index and zone size, the processor can then skip to only scan data items in that zone.

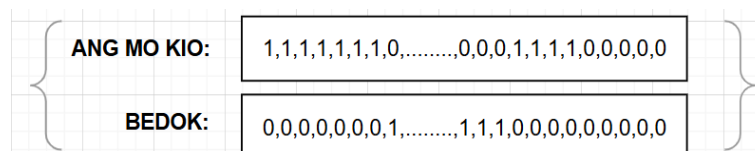
Outcome

Zone maps greatly decreased the number of data items read from the binary **month.store** datafile, reducing the number of I/Os needed to scan and filter the month column. Instead of scanning all 222 833 data items, it only scans a subset of data for the zones that match the criteria.

```
Column: month, Zones read: 6, Data Accessed: 6000
```

3.4 Bitmaps

Instead of storing positions where a value appears, a bitmap index maintains a bit array for each distinct value in a column—where each bit represents whether that value is present in a given row. This allows for highly efficient filtering using bitwise operations, especially when multiple filters are combined using logical operators such as AND and OR. Bitmap indexing is more space-efficient because it uses only a single bit per row to represent the presence or absence of a value. Once the index is created, less I/O calls are required to retrieve and scan the entire bitmap.



Implementation

For our project, Bitmaps can be used for the **TOWN** column due to its low cardinality as there are only 26 distinct towns. Instead of reading and filtering values from the TOWN column store, the processor only retrieves the bitmap index of the selected town.

Outcome

The actual performance improvement observed during query execution was minimal. Since the **TOWN** filter was applied after an already selective filter (e.g. **MONTH**), the number of rows left to process was small. The overhead of bitmap loading and bitwise operations might outweigh its benefits in this case

4 Experiment Result

4.1 Query Execution Result

The results produced by the solution program are validated using Microsoft Excel's Pivot Table. Users can replicate the result by running the main.py script and perform query execution by inputting a valid matriculation number format.

```
Available Matric Numbers:
- 841G
- 392J
Or enter a custom matric number (format: 3 digits + 1 letter, e.g., 123A)
Type 'exit' to go back.

Enter matriculation number: 841G
-----Printing Params -----
Town: WOODLANDS
Start month 2021-04
Next month 2021-05
Column: month, Zones read: 28, Data Accessed: 28000
Number of positions : 4290
Number of positions : 260
Number of positions : 236
Statistical result:
  Year Month      Town      Category      Value
0 2021    04 WOODLANDS      Minimum Price 250000.00
1 2021    04 WOODLANDS      Average Price 476279.29
2 2021    04 WOODLANDS      Standard Deviation of Price 102977.21
3 2021    04 WOODLANDS      Minimum Price per Square Meter 2840.91

Results written to Result/ScanResult_841G.csv
Time Taken: 0.0914 seconds
```

```
Available Matric Numbers:
- 841G
- 392J
Or enter a custom matric number (format: 3 digits + 1 letter, e.g., 123A)
Type 'exit' to go back.

Enter matriculation number: 392J
-----Printing Params -----
Town: CHOA CHU KANG
Start month 2022-09
Next month 2022-10
Column: month, Zones read: 6, Data Accessed: 6000
Number of positions : 4536
Number of positions : 238
Number of positions : 225
Statistical result:
  Year Month      Town      Category      Value
0 2022    09 CHOA CHU KANG      Minimum Price 390000.00
1 2022    09 CHOA CHU KANG      Average Price 557524.60
2 2022    09 CHOA CHU KANG      Standard Deviation of Price 83424.25
3 2022    09 CHOA CHU KANG      Minimum Price per Square Meter 3882.58

Results written to Result/ScanResult_392J.csv
Time Taken: 0.0820 seconds
```

```
results > ScanResult_841G.csv > data
1 Year,Month,Town,Category,Value
2 2021,04,WOODLANDS,Minimum Price,250000.00
3 2021,04,WOODLANDS,Average Price,476279.29
4 2021,04,WOODLANDS,Standard Deviation of Price,102977.21
5 2021,04,WOODLANDS,Minimum Price per Square Meter,2840.91
```

month	(Multiple Items)		
town	WOODLANDS		
area_eligibility	Yes		
Min of resale_price Average of resale_price2 StdDev of resale_price3 Min of price_per_sqm			
	250000	476279.29	102977.21
			2840.91

```
results > ScanResult_392J.csv > data
1 Year,Month,Town,Category,Value
2 2022,09,CHOA CHU KANG,Minimum Price,390000.00
3 2022,09,CHOA CHU KANG,Average Price,557524.60
4 2022,09,CHOA CHU KANG,Standard Deviation of Price,83424.25
5 2022,09,CHOA CHU KANG,Minimum Price per Square Meter,3882.58
6
```

month	(Multiple Items)		
town	CHOA CHU KANG		
area_eligibility	Yes		
Min of resale_price	Average of resale_price2	StdDev of resale_price3	Min of price_per_sqm
390000	557524.60	83424.25	3882.58

4.2 Optimization Result

The experiment conducted included a comparison with different optimization techniques. The first is the base pipeline that loads and applies the filter iteratively. The second is a shared scan pipeline which loads all the columns used and applies the filters in one pass. The following pipeline uses ZoneMaps to load on the month column and apply the filter iteratively. Finally, the last pipeline combines ZoneMaps and SharedScan. The complete logs from running each pipeline is recorded and shown in Appendix B.

Execution Time (s)	392J	841G
Pipeline		
Base Pipeline	0.1957	0.2132
Shared Scan	0.3865	0.4088
Zone Maps	0.0753	0.0898
Bit Maps	0.1985	0.1964
Shared Scan + Zone Maps	0.0704	0.0888

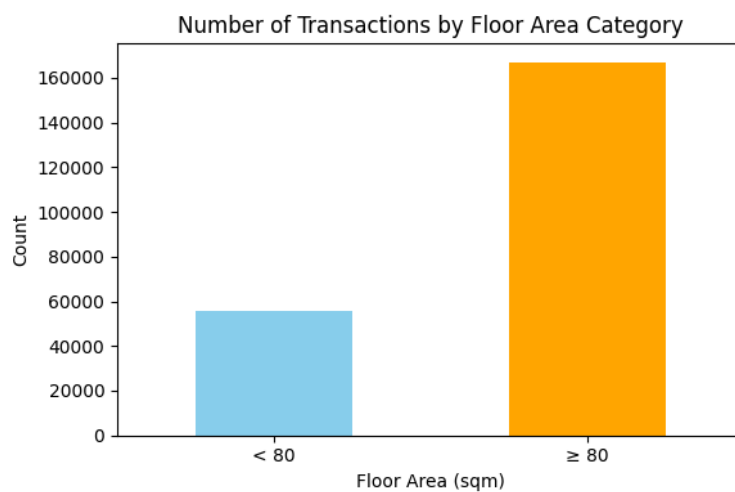
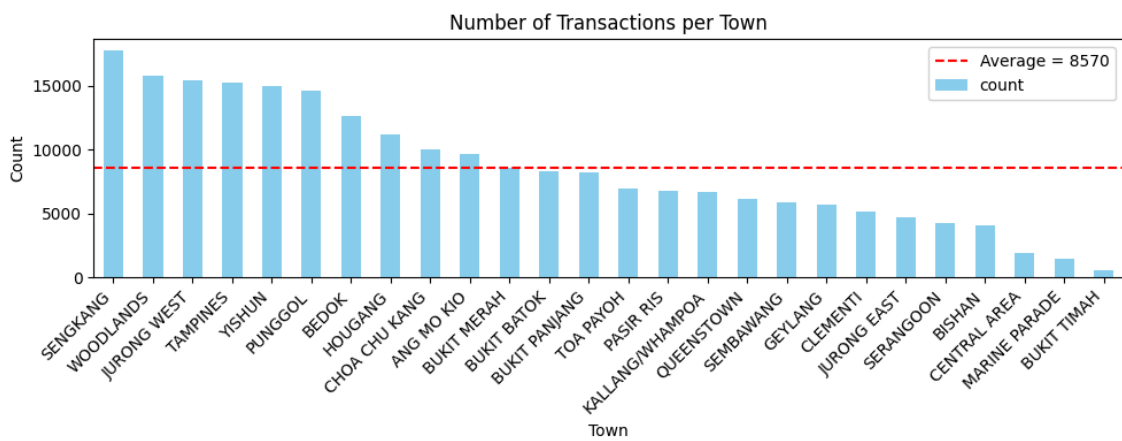
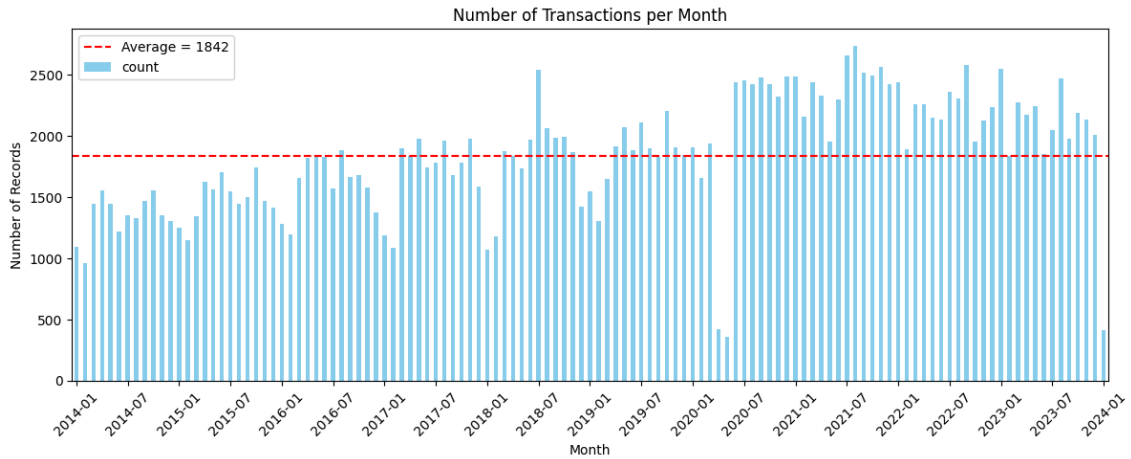
5 Discussion & Conclusion

The statistics produced from the query and pivot table are consistent, showing that the program is able to produce accurate results.

From the optimization result, we can observe that the hybrid approach of Shared Scan and Zone Maps are able to produce the best performance in terms of query execution speed. However, when both approaches are run separately, Shared Scan performed worse than Base Pipeline while Zone Maps still shows impressive improvement. This suggests that Zone Maps is indeed effective in improving the query execution by significantly reducing the number of iterations required.

Meanwhile, the reason why Shared Scan performed poorly could be attributed to the fact that it required full load of all columns compared to the Base Pipeline where only the first column is fully loaded, and the remaining is selectively loaded. Hence, the results suggest that the key performance enhancing factor of a query is reducing the amount of data loaded to complete the execution, which aligns with the idea behind Column Store that aims to eliminate loading of unnecessary data.

Appendix A: Data analysis for Query Sequencing



Appendix B: Experimentation Logs

```
Enter matriculation number: 841G
-----Printing Params -----
Town: WOODLANDS
Start month 2021-04
Next month 2021-05
Base Query Pipeline
Number of positions after datetime Filter: 4290
Number of positions after town filter: 260
Number of positions after area filter: 236
Query Result (Base): 250000.0 476279.29 102977.21 2840.91
Time Taken: 0.2132 seconds

SharedScan Query Pipeline
Number of positions after shared scan : 236
Query Result (SharedScan): 250000.0 476279.29 102977.21 2840.91
Time Taken: 0.4088 seconds

ZoneMap Query Pipeline
Column: month, Zones read: 28, Data Accessed: 28000
Number of positions after datetime Filter: 4290
Number of positions after town filter: 260
Number of positions after area filter: 236
Query Result (ZoneMap): 250000.0 476279.29 102977.21 2840.91
Time Taken: 0.0898 seconds

BitMap Query Pipeline
Number of positions after datetime Filter: 4290
Number of positions After town bitmap filter: 260
Number of positions after area filter: 236
Query Result (BitMap): 250000.0 476279.29 102977.21 2840.91
Time Taken: 0.1964 seconds

ZoneMap + SharedScan Query Pipeline
Column: month, Zones read: 28, Data Accessed: 28000
Number of positions after datetime Filter: 4290
Number of positions after shared scan (ZoneMap) : 236
Query Result (ZoneMap + SharedScan): 250000.0 476279.29 102977.21 2840.91
Time Taken: 0.0888 seconds

Enter matriculation number: 392J
-----Printing Params -----
Town: CHOA CHU KANG
Start month 2022-09
Next month 2022-10
Base Query Pipeline
Number of positions after datetime Filter: 4536
Number of positions after town filter: 238
Number of positions after area filter: 225
Query Result (Base): 390000.0 557524.6 83424.25 3882.58
Time Taken: 0.1957 seconds

SharedScan Query Pipeline
Number of positions after shared scan : 225
Query Result (SharedScan): 390000.0 557524.6 83424.25 3882.58
Time Taken: 0.3865 seconds

ZoneMap Query Pipeline
Column: month, Zones read: 6, Data Accessed: 6000
Number of positions after datetime Filter: 4536
Number of positions after town filter: 238
Number of positions after area filter: 225
Query Result (ZoneMap): 390000.0 557524.6 83424.25 3882.58
Time Taken: 0.0753 seconds

BitMap Query Pipeline
Number of positions after datetime Filter: 4536
Number of positions After town bitmap filter: 238
Number of positions after area filter: 225
Query Result (BitMap): 390000.0 557524.6 83424.25 3882.58
Time Taken: 0.1985 seconds

ZoneMap + SharedScan Query Pipeline
Column: month, Zones read: 6, Data Accessed: 6000
Number of positions after datetime Filter: 4536
Number of positions after shared scan (ZoneMap) : 225
Query Result (ZoneMap + SharedScan): 390000.0 557524.6 83424.25 3882.58
Time Taken: 0.0704 seconds
```

CONTRIBUTION FORM

Group Number - 38

Name	Matriculation Number	Detailed Individual Contribution	Percentage (100% in total)
Tan Wei Yuan	U2122007H	Query Processing Bitmap, Zonemap	50%
Teo Jia Yun	U2120392J	Query Processing Shared Scan, Filter Sequence	50%

Name and Signature from all group members:

Name and Signature of Member 1

Tan Wei Yuan

Tan Wei Yuan

Name and Signature of Member 2

Teo Jia Yun

TJY